BRAC UNIVERSITY

Inspiring Excellence

CSE370 : Database Systems
Project Report
Project Title : Fumble

Group No : __, CSE370 Lab Section : __, Spring 2026		
ID	**Name**	**Contribution**
23201040	Mashrafi Haque	1. Startup creation (company) 2. Co-founder invitation 3. Job listing
23201490	Safwan Ahmed	1. chat/message 2. Social post 3. Investment
23201485	Imtiaz Al Shariar	1. log in/sign up 2. Match (swipe,search) 3. Profile 4. Job Application

Table of Contents

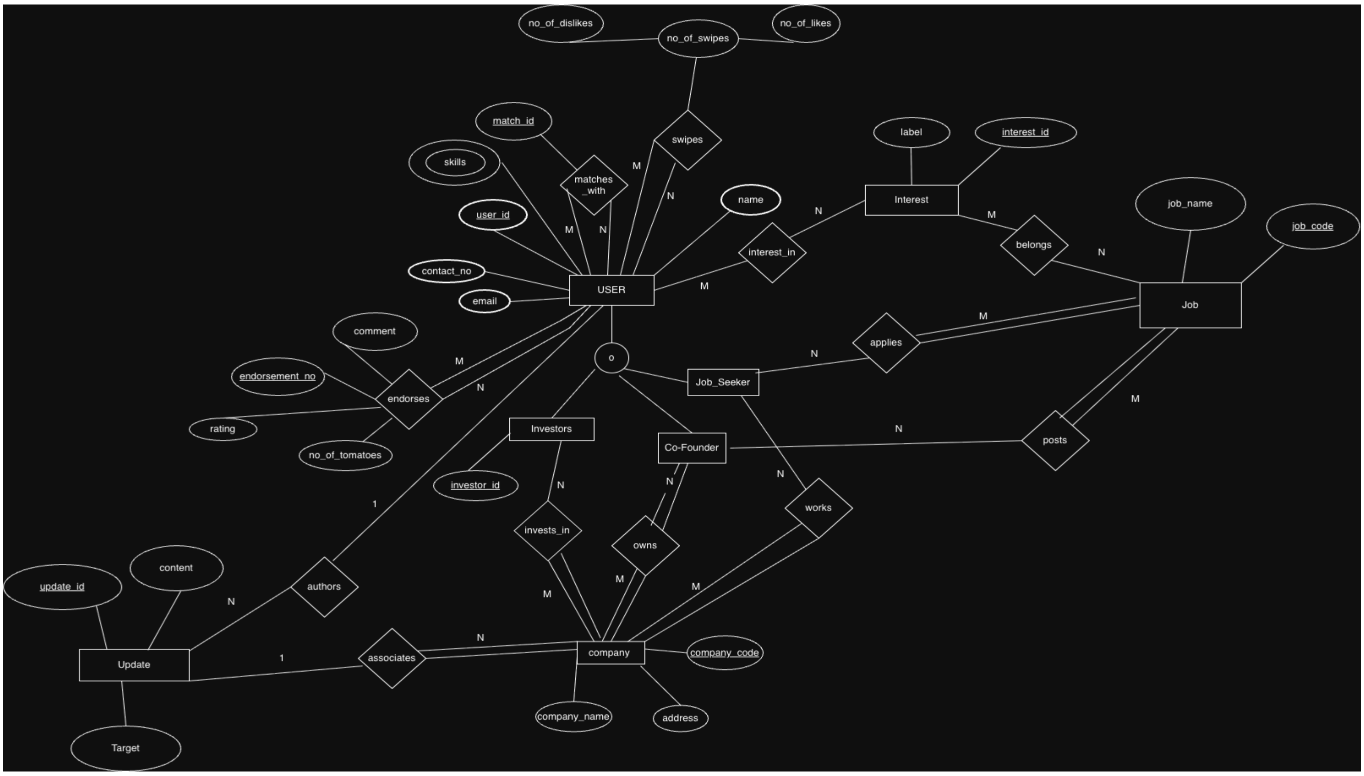
Introduction

Today a lot of people are willing to search for a job, founders are looking for co-founders to work on a project or run a company together. Moreover there are also people who want to invest in a company. The core idea of the project is to create a suitable platform for all of them. The platform that is developed came from the idea of modern dating apps like tinder, bumble. Basically if people are willing to work in a project they will simply right swipe and left swipe otherwise and afterwards a new job post will appear. Using this platform people can search for suitable collaborators or co-founder to work on a project or run a company. There will be options for investments in a company as well. The investors willing to invest can invest. Moreover, there are options for rating and reviewing user profiles and company profiles. This is the rough idea of our project.

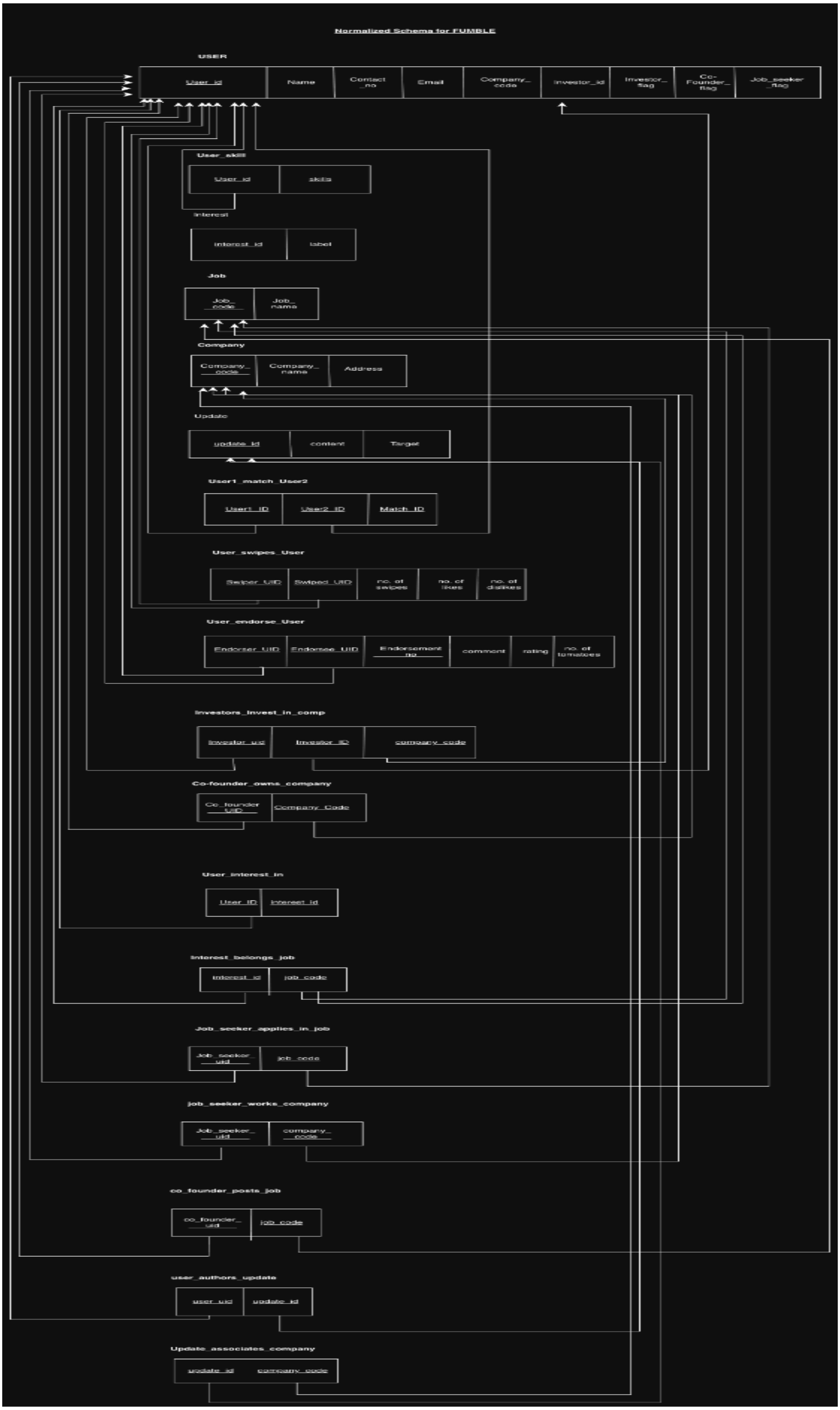
Project Features

ID, Name	Features [3 per member]	
23201040, Mashrafi Haque	Ft 1	Startup creation (company)
	Ft 2	Co-founder invitation
	Ft 3	Job listing
23201490, Safwan Ahmed	Ft 1	chat/message
	Ft 2	Social post
	Ft 3	Investment
23201485, Imtiaz Al Shahriar	Ft 1	log in/sign up
	Ft 2	Match (swipe,search)
	Ft 3	Profile
	Ft 4	Job Application

ER/EER Diagram



SCHEMA DIAGRAM


Normalized Schema for FUMBLE
USER
User_id
Name
Contact_no
Email
Company_code
Investor_id
Investor_flag
Co-Founder_flag
Job_seeker_flag
User_skill
User_id
skills
Interest
interest_id
label
Job
Job_code
Job_name
Company
Company_code
Company_name
Address
Update
update_id
content
Target
User1_match_User2
User1_ID
User2_ID
Match_ID
User_swipes_User
Swiper_UID
Swiped_UID
no. of swipes
no. of likes
no. of dislikes
User_endorse_User
Endorser_UID
Endorsee_UID
Endorsement_no
comment
rating
no. of tomatoes
Investors_invest_in_comp
Investor_uid
Investor_ID
company_code
Co-founder_owns_company
Co_founder_UID
Company_Code
User_interest_in
User_ID
Interest_id
Interest_belongs_job
interest_id
job_code
Job_seeker_applies_in_job
Job_seeker_uid
job_code
job_seeker_works_company
Job_seeker_uid
company_code
co_founder_posts_job
co_founder_uid
job_code
user_authors_update
user_uid
update_id
Update_associates_company
update_id
company_code

Normalization

The entire schema diagram is normalized to 3NF.

Frontend Development

Frontend Development Technology Stack

1. Framework: React 19 with TypeScript
2. Build tool: Vite (fast development and optimized builds)
3. Styling: Tailwind CSS v4 with a custom design system
4. UI components: shadcn/ui
5. Icons: Hugeicons (React integration)
6. Routing: React Router v7
7. State management: React Context API (Auth Provider)

Backend Development

TECHNOLOGY STACK

FastAPI — the main web framework that receives browser requests and sends back answers.

SQLAlchemy — helps our Python code talk to the database using simple objects instead of raw SQL everywhere.

SQLite — a small database file on the computer; good for a class project because it is easy to run.

python-jose and bcrypt — used for login security: passwords are hashed, and users get JWT tokens so the server knows who is logged in.

Pydantic — checks that incoming data looks correct before we use it (for example missing fields or wrong types).

Uvicorn — the program that runs the FastAPI server so the app stays online during testing.

HOW THE BACKEND IS ORGANIZED (LAYERS)

Think of four layers stacked from top to bottom:

Layer 1 — API (FastAPI): URLs, requests, responses, HTTP status codes.

Layer 2 — Business rules: signup/login, matching, important checks like "can this user do this?"

Layer 3 — Data access: SQLAlchemy models and database queries.

Layer 4 — Database (SQLite): where the rows and tables actually live.

MAIN DATABASE PIECES (IN PLAIN WORDS)

User — anyone who signs up (founders and job seekers). A user can have many posts and many messages.

Company — a startup profile. A company can link to multiple founders (users) and post multiple jobs.

Job — one job opening. Each job belongs to one company.

Swipe — a like or pass. It belongs to the person who swiped and points to a user or company they swiped on.

Message — a chat line. Each message has one sender and one receiver (both users).

CompanyInvitation — an invite to join a company as a co-founder. Links a company and two users (inviter and invitee).

Post — a feed post. Each post has one author (user).

KEY FEATURES (WHAT THE APP CAN DO)

1. LOGIN AND ACCOUNTS

How it works: passwords are stored safely (hashed). After login, the app gives a token so the user stays signed in for a while.

Main API paths: POST /auth/register — create an account POST /auth/login — log in and get a token GET /auth/me — see my profile PATCH /auth/me — update my profile

Basic safety ideas: passwords are hashed tokens expire after some time some routes require a logged-in user

2. SWIPING AND MATCHING

Main API paths: POST /swipes — save like or dislike GET /swipes/matches — see mutual likes GET /swipes/history — see past swipes

Simple flow: User A likes User B by swiping right. If User B already liked User A back, they become matched. Matched users can message each other.

3. MESSAGING

Main API paths: POST /messages — send a message GET /messages/{user_id} — view chat with one person GET /messages/conversations — list all chats PATCH /messages/{id}/read — mark a message as read

Simple rules: only matched users can message each other (this blocks random spam chats) tracks unread counts shows recent chats first messages can be marked read automatically when you open the chat

4. COMPANIES AND FOUNDERS

Main API paths: POST /companies — create a company GET /companies/my — list companies I belong to GET /companies/invitable — companies that still accept more founders (under the limit below) POST /companies/{id}/founders/{user_id} — add a founder DELETE /companies/{id}/founders/{user_id} — remove a founder

Important rules: a company cannot have more than five founders (enforced by the backend) only current founders can add or remove founders co-founder invites are tracked with CompanyInvitation records

5. JOBS

Main API paths: POST /companies/{id}/jobs — create a job under a company GET /jobs — list jobs GET /jobs/my — jobs linked to companies I belong to PATCH /jobs/{id} — update a job DELETE /jobs/{id} — delete a job

6. SOCIAL FEED (POSTS)

Main API paths: GET /posts — see the feed POST /posts — create a post GET /posts/{id} — get one post DELETE /posts/{id} — delete my own post

Source Code Repository

https://github.com/mashrafi19/CSE370_Project

Conclusion

Fumble is a small startup matchmaking app. It works a bit like a dating app: people swipe, get matches, and can chat. It helps founders find co-founders and job seekers find startups.

We built a website using Tanstack start and a server FastAPI with a SQLite database. The app lets users sign in, make profiles, post updates, list companies and jobs, swipe, match, and message each other. Fumble is a learning project which shows how database ideas turn into a working app that fits a clear goal: connecting people around startups.